

Pipeline#

<https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html>

Description:

A sequence of data transformers with an optional final predictor. Pipeline allows you to sequentially apply a list of transformers to preprocess the data and, if desired, conclude the sequence with a final predictor for predictive modeling. Intermediate steps of the pipeline must be transformers, that is, they must implement fit and transform methods. The final estimator only needs to implement fit. The transformers in the pipeline can be cached using memory argument. The purpose of the pipeline is to assemble several steps that can be cross-validated together while setting different parameters. For this, it enables setting parameters of the various steps using their names and the parameter name separated by a '__', as in the example below. A step's estimator may be replaced entirely by setting the parameter with its name to another estimator, or a transformer removed by setting it to 'passthrough' or None.

Function Signature

```
class sklearn.pipeline.Pipeline(steps, *,
transform_input=None, memory=None, verbose=False)[source]#
```

Parameters:

stepslist of tuples

transform_inputlist of str, default=None

memorystr or object with the joblib.Memory interface,
default=None

```
class sklearn.pipeline.Pipeline(steps, *,
transform_input=None, memory=None, verbose=False)[source]#
```

Parameters:

stepslist of tuples

transform_inputlist of str, default=None

memorystr or object with the joblib.Memory interface,
default=None

```
decision_function(X, **params)[source]#
```

Parameters:

Xiterable

**paramsdict of string -> object

Returns:

Parameters

****paramsdict of string**

Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them. Added in version 1.4: Only available if `enable_metadata_routing=True`. See [Metadata Routing User Guide](#) for more details.

****paramsdict of str**

If `enable_metadata_routing=False` (default): Parameters passed to the `fit` method of each step, where each parameter name is prefixed such that parameter `p` for step `s` has key `s__p`. If `enable_metadata_routing=True`: Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them. Changed in version 1.4: Parameters are now passed to the `transform` method of the intermediate steps as well, if requested, and if `enable_metadata_routing=True` is set via `set_config`. See [Metadata Routing User Guide](#) for more details.

****paramsdict of str**

If `enable_metadata_routing=False` (default): Parameters to the predict called at the end of all transformations in the pipeline. If `enable_metadata_routing=True`: Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them. Added in version 0.20. Changed in version 1.4: Parameters are now passed to the transform method of the intermediate steps as well, if requested, and if `enable_metadata_routing=True`. See Metadata Routing User Guide for more details. Note that while this may be used to return uncertainties from some models with `return_std` or `return_cov`, uncertainties that are generated by the transformations in the pipeline are not propagated to the final estimator.

****paramsdict of str**

If `enable_metadata_routing=False` (default): Parameters passed to the fit method of each step, where each parameter name is prefixed such that parameter `p` for step `s` has key `s__p`. If `enable_metadata_routing=True`: Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them. Changed in version 1.4: Parameters are now passed to the transform method of the intermediate steps as well, if requested, and if `enable_metadata_routing=True`. See Metadata Routing User Guide for more details.

input_featuresarray

Input features.

paramsmapping of string to any

Parameter names mapped to their values.

Xarray

Data samples, where n_samples is the number of samples and n_features is the number of features. Must fulfill input requirements of last step of pipeline's inverse_transform method.

**paramsdict of str

Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them. Added in version 1.4: Only available if enable_metadata_routing=True. See Metadata Routing User Guide for more details.

****paramsdict of str**

If `enable_metadata_routing=False` (default): Parameters to the `predict` called at the end of all transformations in the pipeline. If `enable_metadata_routing=True`: Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them. Added in version 0.20. Changed in version 1.4: Parameters are now passed to the `transform` method of the intermediate steps as well, if requested, and if `enable_metadata_routing=True` is set via `set_config`. See Metadata Routing User Guide for more details. Note that while this may be used to return uncertainties from some models with `return_std` or `return_cov`, uncertainties that are generated by the transformations in the pipeline are not propagated to the final estimator.

****paramsdict of str**

If `enable_metadata_routing=False` (default): Parameters to the `predict_log_proba` called at the end of all transformations in the pipeline. If `enable_metadata_routing=True`: Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them. Added in version 0.20. Changed in version 1.4: Parameters are now passed to the `transform` method of the intermediate steps as well, if requested, and if `enable_metadata_routing=True`. See Metadata Routing User Guide for more details.

****paramsdict of str**

If `enable_metadata_routing=False` (default): Parameters to the `predict_proba` called at the end of all transformations in the pipeline. If `enable_metadata_routing=True`: Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them. Added in version 0.20. Changed in version 1.4: Parameters are now passed to the `transform` method of the intermediate steps as well, if requested, and if `enable_metadata_routing=True`. See [Metadata Routing User Guide](#) for more details.

sample_weightarray

If not `None`, this argument is passed as `sample_weight` keyword argument to the `score` method of the final estimator.

****paramsdict of str**

Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them. Added in version 1.4: Only available if `enable_metadata_routing=True`. See [Metadata Routing User Guide](#) for more details.

****kwargsdict**

Parameters of this estimator or parameters of estimators contained in steps. Parameters of the steps may be set using its name and the parameter name separated by a '___'.

****paramsdict of str**

Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them. Added in version 1.4: Only available if `enable_metadata_routing=True`. See [Metadata Routing User Guide](#) for more details.

Code Examples

```
>>> from sklearn.svm import SVC
>>> from sklearn.preprocessing import StandardScaler
>>> from sklearn.datasets import make_classification
>>> from sklearn.model_selection import train_test_split
>>> from sklearn.pipeline import Pipeline
>>> X, y = make_classification(random_state=0)
>>> X_train, X_test, y_train, y_test = train_test_split(X, y,
...
random_state=0)
>>> pipe = Pipeline([('scaler', StandardScaler()), ('svc',
SVC())])
>>> # The pipeline can be used as any other estimator
>>> # and avoids leaking the test set into the train set
>>> pipe.fit(X_train, y_train).score(X_test, y_test)
0.88
>>> # An estimator's parameter can be set using '__' syntax
>>> pipe.set_params(svc__C=10).fit(X_train,
y_train).score(X_test, y_test)
0.76
```

Notes & Warnings

NOTE See also `make_pipelineConvenience` function for simplified pipeline construction.

Detailed Documentation

Documentation

A sequence of data transformers with an optional final predictor.

Pipeline allows you to sequentially apply a list of transformers to preprocess the data and, if desired, conclude the sequence with a final predictor for predictive modeling.

Intermediate steps of the pipeline must be transformers, that is, they must implement `fit` and `transform` methods. The final estimator only needs to implement `fit`. The transformers in the pipeline can be cached using `memory` argument.

The purpose of the pipeline is to assemble several steps that can be cross-validated together while setting different parameters. For this, it enables setting parameters of the various steps using their names and the parameter name separated by a `'__'`, as in the example below. A step's estimator may be replaced entirely by setting the parameter with its name to another estimator, or a transformer removed by setting it to `'passthrough'` or `None`.

For an example use case of Pipeline combined with GridSearchCV, refer to [Selecting dimensionality reduction with Pipeline and GridSearchCV](#). The example [Pipelining: chaining a PCA and a logistic regression](#) shows how to grid search on a pipeline using `'__'` as a separator in the parameter names.

Read more in the [User Guide](#).

Added in version 0.5.

List of (name of step, estimator) tuples that are to be chained in sequential order. To be compatible with the scikit-learn API, all steps must define `fit`. All non-last steps must also define `transform`. See [Combining Estimators](#) for more details.

The names of the metadata parameters that should be transformed by the pipeline before passing it to the step consuming it.

This enables transforming some input arguments to fit (other than X) to be transformed by the steps of the pipeline up to the step which requires them. Requirement is defined via metadata routing. For instance, this can be used to pass a validation set through the pipeline.

You can only set this if metadata routing is enabled, which you can enable using `sklearn.set_config(enable_metadata_routing=True)`.

Added in version 1.6.

Used to cache the fitted transformers of the pipeline. The last step will never be cached, even if it is a transformer. By default, no caching is performed. If a string is given, it is the path to the caching directory. Enabling caching triggers a clone of the transformers before fitting. Therefore, the transformer instance given to the pipeline cannot be inspected directly. Use the attribute `named_steps` or `steps` to inspect estimators within the pipeline. Caching the transformers is advantageous when fitting is time consuming. See [Caching nearest neighbors](#) for an example on how to enable caching.

If True, the time elapsed while fitting each step will be printed as it is completed.

Access the steps by name.

Number of features seen during first step fit method.

Names of features seen during first step fit method.

Convenience function for simplified pipeline construction.

Transform the data, and apply `decision_function` with the final estimator.

Call transform of each transformer in the pipeline. The transformed data are finally passed to the final estimator that calls `decision_function` method. Only valid if the final estimator implements `decision_function`.

Data to predict on. Must fulfill input requirements of first step of the pipeline.

Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them.

Added in version 1.4: Only available if `enable_metadata_routing=True`. See Metadata Routing User Guide for more details.

Result of calling `decision_function` on the final estimator.

Fit all the transformers one after the other and sequentially transform the data. Finally, fit the transformed data using the final estimator.

Training data. Must fulfill input requirements of first step of the pipeline.

Training targets. Must fulfill label requirements for all steps of the pipeline.

If `enable_metadata_routing=False` (default): Parameters passed to the `fit` method of each step, where each parameter name is prefixed such that parameter `p` for step `s` has key `s__p`.

If `enable_metadata_routing=True`: Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them.

Changed in version 1.4: Parameters are now passed to the `transform` method of the intermediate steps as well, if requested, and if `enable_metadata_routing=True` is set via `set_config`.

See Metadata Routing User Guide for more details.

Pipeline with fitted steps.

Transform the data, and apply `fit_predict` with the final estimator.

Call `fit_transform` of each transformer in the pipeline. The transformed data are finally passed to the final estimator that calls `fit_predict` method. Only valid if the final estimator implements `fit_predict`.

Training data. Must fulfill input requirements of first step of the pipeline.

Training targets. Must fulfill label requirements for all steps of the pipeline.

If `enable_metadata_routing=False` (default): Parameters to the predict called at the end of all transformations in the pipeline.

If `enable_metadata_routing=True`: Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them.

Added in version 0.20.

Changed in version 1.4: Parameters are now passed to the transform method of the intermediate steps as well, if requested, and if `enable_metadata_routing=True`.

See Metadata Routing User Guide for more details.

Note that while this may be used to return uncertainties from some models with `return_std` or `return_cov`, uncertainties that are generated by the transformations in the pipeline are not propagated to the final estimator.

Result of calling `fit_predict` on the final estimator.

Fit the model and transform with the final estimator.

Fit all the transformers one after the other and sequentially transform the data. Only valid if the final estimator either implements `fit_transform` or `fit` and `transform`.

Training data. Must fulfill input requirements of first step of the pipeline.

Training targets. Must fulfill label requirements for all steps of the pipeline.

If `enable_metadata_routing=False` (default): Parameters passed to the `fit` method of each step, where each parameter name is prefixed such that parameter `p` for step `s` has key `s__p`.

If `enable_metadata_routing=True`: Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them.

Changed in version 1.4: Parameters are now passed to the `transform` method of the intermediate steps as well, if requested, and if `enable_metadata_routing=True`.

See Metadata Routing User Guide for more details.

Get output feature names for transformation.

Transform input features using the pipeline.

Transformed feature names.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

A `MetadataRouter` encapsulating routing information.

Get parameters for this estimator.

Returns the parameters given in the constructor as well as the estimators contained within the steps of the Pipeline.

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

Apply `inverse_transform` for each step in a reverse order.

All estimators in the pipeline must support `inverse_transform`.

Data samples, where `n_samples` is the number of samples and `n_features` is the number of features. Must fulfill input requirements of last step of pipeline's `inverse_transform` method.

Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them.

Added in version 1.4: Only available if `enable_metadata_routing=True`. See Metadata Routing User Guide for more details.

Inverse transformed data, that is, data in the original feature space.

Access the steps by name.

Read-only attribute to access any step by given name. Keys are steps names and values are the steps objects.

Transform the data, and apply predict with the final estimator.

Call transform of each transformer in the pipeline. The transformed data are finally passed to the final estimator that calls predict method. Only valid if the final estimator implements predict.

Data to predict on. Must fulfill input requirements of first step of the pipeline.

If enable_metadata_routing=False (default): Parameters to the predict called at the end of all transformations in the pipeline.

If enable_metadata_routing=True: Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them.

Added in version 0.20.

Changed in version 1.4: Parameters are now passed to the transform method of the intermediate steps as well, if requested, and if enable_metadata_routing=True is set via set_config.

See Metadata Routing User Guide for more details.

Note that while this may be used to return uncertainties from some models with return_std or return_cov, uncertainties that are generated by the transformations in the pipeline are not propagated to the final estimator.

Result of calling predict on the final estimator.

Transform the data, and apply predict_log_proba with the final estimator.

Call transform of each transformer in the pipeline. The transformed data are finally passed to the final estimator that calls predict_log_proba method. Only valid if the final estimator implements predict_log_proba.

Data to predict on. Must fulfill input requirements of first step of the pipeline.

If `enable_metadata_routing=False` (default): Parameters to the `predict_log_proba` called at the end of all transformations in the pipeline.

If `enable_metadata_routing=True`: Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them.

Added in version 0.20.

Changed in version 1.4: Parameters are now passed to the `transform` method of the intermediate steps as well, if requested, and if `enable_metadata_routing=True`.

See Metadata Routing User Guide for more details.

Result of calling `predict_log_proba` on the final estimator.

Transform the data, and apply `predict_proba` with the final estimator.

Call `transform` of each transformer in the pipeline. The transformed data are finally passed to the final estimator that calls `predict_proba` method. Only valid if the final estimator implements `predict_proba`.

Data to predict on. Must fulfill input requirements of first step of the pipeline.

If `enable_metadata_routing=False` (default): Parameters to the `predict_proba` called at the end of all transformations in the pipeline.

If `enable_metadata_routing=True`: Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them.

Added in version 0.20.

Changed in version 1.4: Parameters are now passed to the transform method of the intermediate steps as well, if requested, and if `enable_metadata_routing=True`.

See Metadata Routing User Guide for more details.

Result of calling `predict_proba` on the final estimator.

Transform the data, and apply score with the final estimator.

Call transform of each transformer in the pipeline. The transformed data are finally passed to the final estimator that calls score method. Only valid if the final estimator implements score.

Data to predict on. Must fulfill input requirements of first step of the pipeline.

Targets used for scoring. Must fulfill label requirements for all steps of the pipeline.

If not None, this argument is passed as `sample_weight` keyword argument to the score method of the final estimator.

Parameters requested and accepted by steps. Each step must have requested certain metadata for these parameters to be forwarded to them.

Added in version 1.4: Only available if `enable_metad`